🎥 Explicación del `EvaluacionController.php`

Este archivo es el **controller principal** del sistema de Evaluación Académica.

Su función es **controlar lo que hace la aplicación** cuando el usuario entra a una página, envía un formulario o quiere ver información.

🧱 Encabezado del archivo

```
namespace app\controllers;
```

Aquí se indica que este archivo pertenece a la carpeta **controllers** del proyecto.

```
use Yii;
use yii\web\Controller;
use yii\web\NotFoundHttpException;
use app\models\Evaluacion;
```

Estas líneas importan las herramientas necesarias:

- **Yii** → para acceder al framework
- **Controller** → para poder crear un controlador
- **NotFoundHttpException** → para mostrar error 404
- **Evaluacion** → el modelo que conecta con la base de datos

🧩 Clase principal

```
class EvaluacionController extends Controller
```

Esta clase es el **controlador de evaluaciones**.

Todo lo relacionado con crear, listar y ver evaluaciones pasa por aquí.

📄 actionIndex()

```
public function actionIndex()
```

Esta función:

- Obtiene **todas las evaluaciones** de la base de datos
- Las ordena por ID de la más reciente a la más antigua
- Las envía a la vista `index.php`

En palabras simples:

"Muestra la lista de todas las evaluaciones registradas."

➕ actionCreate()

```
public function actionCreate()
```

Esta función:

- Crea un nuevo objeto `Evaluacion`
- Recibe los datos del formulario
- Guarda la información en la base de datos
- Redirige al listado cuando todo sale bien

En palabras simples:

"Se encarga de crear una nueva evaluación y guardarla."

📊 actionGrafico($id)

```
public function actionGrafico($id)
```

Esta función:

- Busca una evaluación específica por su ID
- Verifica que exista

- Si no existe, muestra un error
- Si existe, muestra la vista del gráfico

En palabras simples:

“Muestra el gráfico del rendimiento del estudiante seleccionado.”

🚨 Validación de error

```
if ($model === null) {
    throw new NotFoundHttpException('Estudiante no encontrado');
}
```

Esto evita errores:

- Si alguien intenta ver un estudiante que no existe
- Yii muestra una página 404 correctamente

🧠 Resumen para cerrar el video

“Este controller es el cerebro del sistema.

Se encarga de mostrar las evaluaciones, crear nuevas y mostrar el gráfico del desempeño académico.

Sin este archivo, la aplicación no sabría qué hacer cuando el usuario navega por el sistema.”

🎥 Explicación del `ApiController.php`

Este archivo es un **controlador tipo API**.

A diferencia del controller normal, **no muestra páginas**, sino que **devuelve datos en formato JSON**, para que puedan ser usados por otras aplicaciones, como gráficos, dashboards o apps externas.

🧱 Encabezado del archivo

```
namespace app\controllers;
```

Indica que este archivo pertenece a la carpeta **controllers**.

```
use Yii;
use yii\web\Controller;
use yii\web\Response;
use app\models\Evaluacion;
```

Aquí se importan:

- **Yii** → núcleo del framework
- **Controller** → para crear el controlador
- **Response** → para devolver respuestas JSON
- **Evaluacion** → modelo que conecta con la base de datos

🧩 Clase ApiController

```
class ApiController extends Controller
```

Esta clase maneja **servicios web**, no vistas.

Sirve para que **otras plataformas consuman los datos del sistema**.

🔓 Desactivar CSRF

```
public $enableCsrfValidation = false;
```

Esto se hace porque:

- Las APIs no usan formularios
- Permite que apps externas se conecten sin bloqueo

⚙️ beforeAction()

```
public function beforeAction($action)
```

Este método se ejecuta **antes de cualquier acción del controller**.

Aquí se hacen varias cosas importantes:

📦 Respuesta en JSON

```
Yii::$app->response->format = Response::FORMAT_JSON;
```

Esto asegura que **todas las respuestas** sean en formato JSON, ideal para APIs.

🌐 Configuración CORS

```
$headers->set('Access-Control-Allow-Origin', '*');
```

Esto permite que:

- Otras páginas web
- CodeSandbox
- Replit
- O aplicaciones externas

puedan consumir la API sin errores de seguridad.

En palabras simples:

“Le damos permiso a cualquier origen para acceder a los datos.”

✋ Peticiones OPTIONS (preflight)

```
if (Yii::$app->request->isOptions) {
    Yii::$app->response->statusCode = 200;
```

```
    return false;
}
```

Esto evita errores cuando el navegador **verifica permisos antes de consumir la API**.

📄 actionLista()

```
public function actionLista()
```

Esta es la **ruta principal de la API**.

Hace lo siguiente:

- Obtiene todas las evaluaciones
- Las ordena por fecha (más recientes primero)
- Las convierte en arreglo
- Las devuelve en JSON

📤 Respuesta final

```
return [
    'success' => true,
    'count' => count($items),
    'data' => $items,
];
```

Esto devuelve:

- Si la petición fue exitosa
- Cuántos registros hay
- Los datos completos

En palabras simples:

“Esta función entrega todas las evaluaciones para ser usadas en gráficos u otras aplicaciones.”

🧠 Cierre ideal para el video

“Este ApiController permite que el sistema de evaluaciones se conecte con otras plataformas.

Excelente, esto ya es **la parte visual del proyecto (las V de MVC)** ✌️

Te dejo la **explicación humanizada, clara y lista para video** de **las tres vistas**: `create`, `grafico` e `index`. No es técnica pesada y se entiende fácil.

🎥 Explicación de las VISTAS (Views) en Yii

Las **views** son las encargadas de **mostrar la información al usuario**.

No hacen cálculos ni consultas a la base de datos, solo **presentan los datos** que les envía el controller.

📄 Vista `create.php`

👉 Registrar una evaluación

Esta vista muestra el **formulario para ingresar los datos del estudiante**.

```
use yii\helpers\Html;
use yii\widgets\ActiveForm;
```

Se usan estas clases para:

- Crear formularios seguros
- Evitar errores de seguridad como inyección de código

🆔 Campo ID

```
$form->field($model, 'id')
```

Aquí el usuario **digita manualmente el ID del estudiante**, ya que:

- El ID **no es automático**
- Se controla desde el sistema académico

👤 Nombre y notas

Los campos:

- Nombre
- Nota 1
- Nota 2
- Nota 3

Permiten ingresar las calificaciones del estudiante, con límites de 0 a 100 para evitar errores.

📊 Promedio y nivel

```
'readonly' => true,
'disabled' => true
```

Estos campos:

- **No se escriben a mano**
- Se calculan automáticamente en el modelo
- Solo se muestran al usuario

Esto garantiza que **los resultados sean confiables**.

💾 Botones

- **Guardar** → envía el formulario
- **Volver** → regresa al listado

📊 Vista `grafico.php`

👉 Gráfico de rendimiento académico

Esta vista muestra un **gráfico visual del desempeño del estudiante**.

📌 Datos del estudiante

```
$model->nota1
$model->nota2
$model->nota3
$model->promedio
```

Se toman los datos del modelo y se convierten a números para usarlos en JavaScript.

📈 Chart.js

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

Se usa **Chart.js**, una librería externa, para:

- Dibujar gráficos
- Visualizar el progreso del estudiante

📉 Gráfico de línea

El gráfico muestra:

- Nota 1
- Nota 2
- Nota 3
- Promedio

Esto permite ver **si el estudiante mejoró o bajó su rendimiento** de forma clara y visual.

🔙 Botón Volver

Permite regresar al listado sin perder información.

📋 Vista `index.php`

👉 Listado general de evaluaciones

Esta vista muestra **todas las evaluaciones registradas en el sistema**.

📃 Tabla de datos

Se presentan:

- ID
- Nombre
- Notas
- Promedio
- Nivel
- Fecha

Todo viene directamente desde la base de datos.

⚠️ Validación

```
if (empty($evaluaciones))
```

Si no hay datos, el sistema muestra un mensaje claro:

“No hay evaluaciones registradas.”

📊 Botón Gráfico

```
Html::a('Gráfico', ['grafico', 'id' => $e->id])
```

Este botón:

- Envía el ID del estudiante
- Abre el gráfico individual
- Conecta el index con la vista gráfica

🧠 Cierre perfecto para el video

“Las vistas se encargan de mostrar la información al usuario.

El formulario permite registrar evaluaciones, el index muestra todas y el gráfico ayuda a analizar el rendimiento académico de forma visual.

Gracias a esta separación, el sistema es claro, ordenado y fácil de mantener.”